# SoundSense — Comprehensive System Guide

Complete Technical Blueprint: Signal Ingestion, YAMNet/Whisper ML, GCC-PHAT 360° DOA, WebSocket Streaming & Safety Engine

| | |
|---|---|
| **Author / Project:** SoundSense Research Team | **Target Device:** Standalone Smartphone (Samsung S23 FE / Mobile Web) |
| **Backend Stack:** Python FastAPI, WebSocket, SciPy, NumPy | **ML Models:** Google YAMNet (AudioSet), OpenAI Whisper ASR |

## 1. Executive Summary & Core Philosophy

**SoundSense** is an end-to-end assistive sensory substitution system engineered specifically for deaf and hard-of-hearing individuals. Unlike legacy assistive tools that require dedicated external multi-microphone hardware arrays or companion smartwatches, SoundSense turns a **standard smartphone** into an all-in-one assistive acoustic radar. The system continuously listens to environmental soundwaves, classifies acoustic hazards, transcribes live speech, estimates 360° spatial direction-of-arrival (DOA), evaluates situational danger scores, and presents life-saving sensory alerts via visual radar and haptic vibrations.

**The 3-Second Elevator Pitch:** *'SoundSense transforms your mobile phone into an AI-powered acoustic radar. It captures ambient sound, routes it through deep learning models for classification and speech transcription, pinpoints the 360° direction of the noise using phase correlation and chassis diffraction, and triggers instant visual and haptic alerts if you are in danger.'*

## 2. High-Level Architectural Flow

The architecture is organized as a high-throughput, low-latency client-server pipeline designed for sub-250ms reaction times:

| Phase | Component | Function & Technical Mechanism |
|---|---|---|
| **1. Capture** | Mobile Web Audio API | Captures 48 kHz stereo PCM audio via dual-channel MediaStreamSource & ScriptProcessor; accumulates ~250ms chunks (12,288 samples). |
| **2. Transport** | WebSocket (/ws/audio-stream) | Streams interleaved 16-bit PCM (base64-encoded) over persistent low-latency full-duplex WebSocket connection. |
| **3. Preprocess** | FastAPI AudioPreprocessor | Decodes PCM; applies 4th-order Butterworth High-Pass filter (80 Hz cutoff); computes RMS amplitude and calibrated dB SPL. |
| **4. Ingestion** | Dual-Stage RMS Gate | Switches between 16 kHz low-power idle mode and 48 kHz active inference mode to prevent server compute saturation. |
| **5. ML Routing** | YAMNet Classifier | Resamples to 16 kHz; computes 64-band log-mel spectrogram; classifies into 521 AudioSet categories; routes to Branch A (Speech) or Branch B (Acoustic Hazard). |
| **6. Branch A** | Whisper Speech ASR | If speech/shout detected: executes Whisper transformer for real-time natural language speech-to-text transcription. |
| **7. Branch B** | GCC-PHAT 360° DOA Engine | Cross-Power Spectrum with PHAT whitening computes TDOA (τ); resolves front/back cone-of-confusion using chassis diffraction index. |
| **8. Safety** | DangerEngine & Hysteresis | Calculates composite Hazard Score D in [0, 100]; applies 3s cooldown & 1.5s hysteresis; triggers 10s SOS countdown if D >= 90. |
| **9. Presentation** | 360° Radar Compass UI | Presents direction needle, priority badges (P0/P1/P2/P3), live dB SPL metering, haptic patterns, and live transcripts. |

# 3. Machine Learning Models & Inference Pipeline

SoundSense utilizes a dual-model artificial intelligence architecture to separate speech comprehension from environmental situational awareness:

## 3.1 YAMNet (Acoustic Scene & Event Classifier)

**Model Architecture:** YAMNet is a lightweight deep convolutional neural network based on the **MobileNet-v1** architecture, specifically optimized for edge and mobile deployment. It employs depthwise separable convolutions that decouple spatial filtering from channel-wise feature mixing, slashing parameter count by over 80% compared to standard ResNet/VGG architectures.

- **Input Representation:** 16 kHz mono waveform is sliced into 25 ms Hann-windowed frames with a 10 ms hop size. A 64-band log-mel spectrogram is computed across the 125 Hz – 7500 Hz frequency range, mimicking human cochlear frequency sensitivity.
- **Acoustic Ontology & Routing:** YAMNet produces softmax probabilities across 521 AudioSet classes. SoundSense maps these into an assistive ontology of 37 critical categories with predefined severity ratings (P0 Critical, P1 Caution, P2 Awareness, P3 Informational).
- **Confidence Gating:** The orchestrator enforces a 70% confidence threshold (C >= 0.70) to suppress environmental false alarms while maintaining high recall on critical alerts (sirens, car horns, dog barks, fire alarms).

## 3.2 OpenAI Whisper (Natural Language ASR)

When YAMNet flags that the incoming sound belongs to human vocalization (speech, screaming, shouting, crowd chatter), the pipeline routes the audio to **Branch A (Whisper ASR)**. Whisper is an encoder-decoder transformer trained on 680,000 hours of multilingual, acoustically diverse audio. It performs noise-robust speech recognition, outputting transcription text, speaker confidence, and finality tokens to the mobile UI in real-time, allowing deaf users to read public transit announcements or shouted warnings instantly.

# 4. Spatial Audio Localization & 360° GCC-PHAT Engine

To tell a deaf user *where* a danger is approaching from, SoundSense implements physical **Time Difference of Arrival (TDOA)** estimation using the **Generalized Cross-Correlation with Phase Transform (GCC-PHAT)** algorithm.

## 4.1 Mathematical Formulation of GCC-PHAT

Given Left channel signal $x_L(t)$ and Right channel signal $x_R(t)$ recorded at sample rate $f_s = 48$ kHz with physical microphone spacing $d = 0.14$ m and speed of sound $c = 343$ m/s:

1. **Discrete Fourier Transforms:** $X_L(f) = \text{FFT}(x_L)$ and $X_R(f) = \text{FFT}(x_R)$
2. **Cross-Power Spectrum:** $G_{RL}(f) = X_R(f) \cdot \text{conj}(X_L(f))$
3. **PHAT Whitening Transform:** $\psi(f) = G_{RL}(f) / (|G_{RL}(f)| + \varepsilon)$. This equalizes frequency magnitudes, stripping acoustic room reverberation and keeping only pure phase differences.
4. **Inverse FFT:** $cc(\tau) = \text{IFFT}(\psi(f))$. The peak lag $n_{peak} = \text{argmax}(cc(\tau))$ corresponds to the inter-channel delay in sample counts.
5. **TDOA & Angular Arrival:** $\tau = n_{peak} / f_s$. The acoustic angle $\theta = \arccos( -c \cdot \tau / d )$.

## 4.2 Resolving the Front-vs-Back 'Cone of Confusion'

A fundamental limitation of two collinear microphones is the **Cone of Confusion**: a sound in Front (Ahead, 90°) and a sound directly Behind (Back, 270°) both produce identical zero delay ($\tau = 0$). SoundSense solves this physical ambiguity through **Chassis Acoustic Diffraction & Spectral Shadowing Analysis**:

- **Phone Chassis Obstacle:** A smartphone body (glass back, internal battery, metal chassis) acts as an acoustic barrier for sounds coming from behind the phone.

**High-Frequency Shadowing:** High frequencies (> 3.5 kHz, wavelength < 10 cm) cannot wrap around the phone chassis without undergoing substantial diffraction loss (6 dB to 12 dB drop). Conversely, low/mid frequencies (500 Hz - 2.5 kHz) diffract easily.

**Diffraction Ratio Metric:** The orchestrator calculates the ratio R = E_high / E_mid. If R < 0.085, the sound source is classified in the **Rear Hemisphere**, reflecting the angle across the horizontal axis: θ_rear = 360° - θ_front. This enables true 360° localization: South (270°), South-West (225°), and South-East (315°).

## 5. Safety Assessment & Emergency SOS Engine

Raw sound classification is not enough — a siren 2 blocks away is awareness, but a car horn 2 meters away is life-threatening. The **DangerEngine** evaluates urgency through dynamic scoring, temporal hysteresis, and automated incident escalation:

**Composite Danger Score:** D = w_base · S_type + w_vol · f(dB_SPL) + w_conf · C + w_ctx · Context. Scores range from 0 to 100 and map to 4 danger tiers: **SAFE (0-39)**, **AWARENESS (40-69)**, **CAUTION (70-89)**, and **DANGER (90-100)**.

**Temporal Hysteresis & Cooldown:** To prevent rapid alert flickering, alerts require a cooldown of 3.0s per category, and danger score de-escalation requires a 1.5s delay with a -15 point drop.

**Automated SOS Trigger:** If a critical danger state (D >= 90) is sustained for 10 consecutive seconds without manual dismissal, an automated 10-second SOS countdown is triggered, notifying emergency contacts and logging the incident.

## 6. Full-Duplex WebSocket Protocol & Streaming

Communication runs over a bidirectional WebSocket endpoint at /ws/audio-stream. Unlike REST polling, WebSockets eliminate HTTP handshake overhead and enable true real-time sub-100ms bidirectional event streaming:

| Direction | Message Type | Payload Attributes | Description |
|---|---|---|---|
| **Client -> Server** | audio_chunk | data (base64 PCM), fs (48k), channels (2), seq | Continuously streams 250ms interleaved 16-bit stereo PCM audio. |
| **Client -> Server** | synthetic_trigger | sound_class, delay_samples, db_spl, is_rear | Simulator trigger for testing directional angles without hardware. |
| **Client -> Server** | config_update | rms_threshold, mic_spacing_m, context_score | Updates client sensitivity and acoustic calibration thresholds. |
| **Server -> Client** | connected | session_id, server_time, message | Handshake confirmation establishing active audio session. |
| **Server -> Client** | status | state (active/idle), rms, db_spl, timestamp | Live RMS amplitude and dB SPL metering emitted every 250ms. |
| **Server -> Client** | sound_event | class, display_name, angle_deg, compass_zone, priority | Emitted when classified hazard meets confidence and cooldown gates. |
| **Server -> Client** | transcription | text, speaker, confidence, is_final | Emitted when Whisper ASR transcribes human speech / announcements. |
| **Server -> Client** | danger_update | score (0-100), level, factors, sos_auto_triggered | Real-time safety assessment update driving UI gauge & warnings. |

## 7. How to Explain SoundSense (Presentation Guide)

Use this structured talking track when explaining SoundSense to recruiters, professors, or hackathon judges:

**Q: What makes SoundSense novel compared to existing sound recognition apps?**
*A: Standard accessibility apps only do mono sound classification — they say 'a horn honked', but can't tell you where it came from. SoundSense provides real-time 360° spatial localization using GCC-PHAT and chassis diffraction on a standard smartphone, plus an intelligent danger evaluation engine that prevents alert fatigue through hysteresis and auto-triggers emergency SOS if high hazard persists.*

**Q: How does direction detection work on a phone without an external mic array?**
*A: Smartphones like the Samsung S23 FE possess top and bottom microphones spaced ~14cm apart. When held in landscape mode, sound hits one microphone before the other. We apply GCC-PHAT to compute the cross-power spectrum and inverse Fourier transform to find the inter-channel delay ($\tau$). To resolve front versus back ambiguity, our algorithm evaluates high-frequency diffraction shadowing caused by the phone's physical chassis.*

**Q: How do you handle low-latency streaming without draining battery?**
*A: We use a dual-stage ingestion architecture. When ambient acoustics are quiet, the system operates in a low-power 16 kHz idle monitoring mode. Only when an acoustic surge exceeds the calibrated RMS threshold does the system transition to 48 kHz active inference. We batch audio into ~250ms chunks streamed over persistent WebSockets, keeping round-trip latency under 300ms while eliminating HTTP polling overhead.*

---

**SoundSense Assistive System** — Developed with Python, FastAPI, YAMNet, Whisper & Web Audio API.